Assignment No. 5

Name : Mohit Sojal **PRN :** 124B1F027

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/mohitsojal1/DAA--Assignment/tree/main

Multistage Graph

1. Problem Statement

A logistics company, SwiftCargo, specializes in delivering packages across multiple cities. To optimize its delivery process, the company divides its transportation network into multiple stages consisting of warehouses, transit hubs, and final delivery points.

The objective is to determine the minimum-cost delivery route from a source location to a destination while following the predefined stages. The transportation network is represented as a directed, weighted multistage graph, where vertices represent locations and edges represent routes with associated costs.

The system must use Dynamic Programming to calculate optimal routes efficiently, support multiple delivery requests, and allow route recalculation when conditions such as traffic, weather, or road closures change.

Tasks:

1. Model the transportation network as a directed, weighted multistage graph.
2. Implement a Dynamic Programming algorithm to find the minimum-cost route.
3. Reconstruct the optimal path from the source to the destination.
4. Support changes in route costs and recalculate the optimal route.
5. Process multiple delivery requests efficiently.

2. Course Objectives

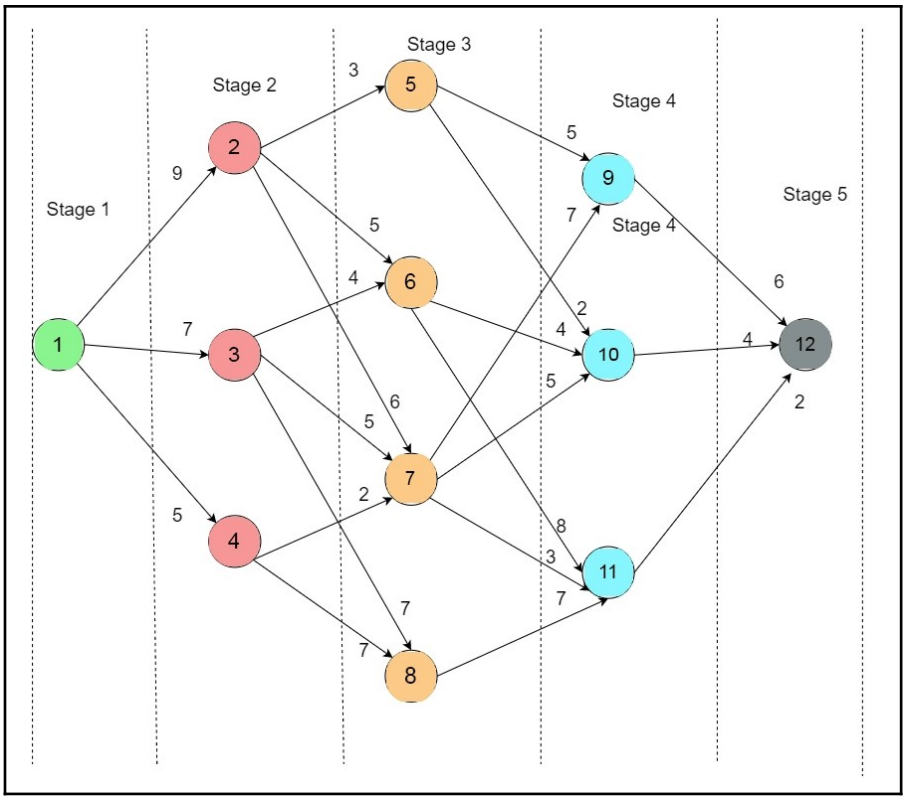
1. To understand the computational complexity of various algorithms.
2. To select appropriate algorithm design strategies for solving real-world problems.

3. Course Outcomes

1. Analyze the asymptotic performance of algorithms.
2. Generate optimal solutions by applying the Dynamic Programming strategy.

4. Theory

4.1 Multistage Graph



A multistage graph is a directed, weighted graph in which the vertices are divided into a number of disjoint stages. The stages are represented as (V_1, V_2, \ldots, V_k).

In a standard multistage graph, edges connect vertices in one stage to vertices in the immediately succeeding stage. The source vertex belongs to the first stage, and the destination vertex belongs to the final stage.

Each edge has an associated weight representing the cost, distance, or time required to travel between two locations. The objective is to find a path from the source to the destination with the minimum total cost.

4.2 Structure of a Multistage Graph

Consider a multistage graph divided into five stages:

- **Stage 1:** (V_1 = {A}) — Source node
- **Stage 2:** (V_2 = {B,C,D,E})
- **Stage 3:** (V_3 = {F,G,H})
- **Stage 4:** (V_4 = {I,J,K})
- **Stage 5:** (V_5 = {L}) — Destination node

The directed edges connect consecutive stages, and each edge has a weight representing its travel cost.

A valid path begins at the source node (A), passes through the intermediate stages in order, and terminates at the destination node (L). The objective is to select the path with the minimum sum of edge weights.

4.3 Applications of Multistage Graphs

Multistage graphs are useful in several real-world applications:

1. **Logistics:** Finding cost-effective routes for transporting packages through warehouses and distribution centres.
2. **Network Routing:** Selecting efficient paths for transmitting data through network layers or intermediate routers.
3. **Manufacturing:** Optimizing costs across a sequence of production stages.
4. **Speech Recognition:** Evaluating possible sequences of words or states.
5. **Finance:** Optimizing investment decisions across multiple time periods.

5. Dynamic Programming

5.1 Definition

Dynamic Programming (DP) is an algorithmic technique used to solve complex problems by dividing them into smaller subproblems, solving each subproblem, and storing its result to avoid unnecessary repeated calculations.

Dynamic Programming is particularly effective when a problem has optimal substructure and overlapping subproblems.

5.2 Key Properties

1. Optimal Substructure

A problem has optimal substructure when its optimal solution can be constructed from optimal solutions to smaller subproblems.

In a multistage graph, the minimum cost from a vertex to the destination depends on the edge costs to its successor vertices and the minimum costs from those successors to the destination.

2. Overlapping Subproblems

Overlapping subproblems occur when the same smaller problems are encountered repeatedly.

Dynamic Programming stores previously calculated results so that they can be reused instead of recalculated.

5.3 Why Dynamic Programming Is Suitable

Dynamic Programming is suitable for multistage graphs because:

- The graph is directed and acyclic under the standard multistage structure.
- The optimal route can be calculated using the optimal routes from successor vertices.
- Each vertex's minimum remaining cost can be calculated and stored once.
- A separate array can record the next vertex on the optimal path.
- The algorithm avoids enumerating every possible route.

A greedy algorithm may not always find the globally optimal route because a locally cheapest edge does not necessarily lead to the cheapest complete path. Dynamic Programming considers the remaining cost to the destination when selecting a route.

6. Working of Dynamic Programming in a Multistage Graph

The algorithm calculates the minimum cost by processing the graph backward, starting from the destination.

Step 1: Initialize the destination

Set the minimum cost of the destination to zero because no additional travel is required once it has been reached.

$$
\text{Cost[destination]} = 0
$$

Step 2: Process vertices in reverse stage order

Starting from the stage immediately before the destination, process each vertex in reverse stage order.

Step 3: Calculate the minimum cost

For each vertex (i), consider every outgoing edge from (i) to a successor vertex (j). Calculate the total cost of travelling to (j) and then following the optimal route from (j) to the destination.

$$
\text{Cost[i]} = \min_{(i,j)\in E}\{w(i,j)+\text{Cost[j]}\}
$$

Where:

- (Cost[i]) is the minimum cost from vertex (i) to the destination.
- (w(i,j)) is the weight of the edge from vertex (i) to vertex (j).
- (Cost[j]) is the minimum cost from vertex (j) to the destination.

If a vertex has no valid route to the destination, its cost remains infinity.

Step 4: Store the next vertex

Whenever a lower-cost route is found, store the corresponding successor vertex in the next[] array.

Step 5: Determine the minimum cost

After processing all stages, the value stored in Cost[source] gives the minimum cost from the source to the destination, provided a valid route exists.

Step 6: Reconstruct the path

Starting from the source, repeatedly follow the vertices stored in the next[] array until the destination is reached.

7. Algorithm

Algorithm: Multistage Graph Using Dynamic Programming

Input: A directed, weighted multistage graph (G=(V,E)), its stage assignments, a source vertex (s), and a destination vertex (t).

Output: The minimum route cost and the corresponding path.

1. Initialize the cost[] array with infinity.
2. Initialize the next[] array to indicate that no successor has been selected.
3. Set cost[destination] = 0.
4. Process the stages in reverse order, from the stage before the destination to the first stage.
5. For every vertex (i) in the current stage:
    - Examine every outgoing edge ((i,j)).
    - Calculate weight(i,j) + cost[j] when the successor has a valid route to the destination.
    - If this value is smaller than cost[i], update cost[i] and set next[i] = j.
6. If the source has no valid route to the destination, report that no path exists.
7. Otherwise, start at the source and follow the next[] array to reconstruct the optimal path.
8. Display the minimum cost and the reconstructed path.

8. Pseudocode

```
MULTISTAGE-DP(G, stages, source, destination)

  for each vertex v in G:
    cost[v] = INFINITY
    next[v] = NULL

  cost[destination] = 0

  for each stage k in reverse order,
    excluding the final stage:

    for each vertex u in stage k:

      for each edge (u, v) with weight w:

        if cost[v] != INFINITY:

          currentCost = w + cost[v]

          if currentCost < cost[u]:
            cost[u] = currentCost
            next[u] = v
```

```
if cost[source] == INFINITY:
   report "No valid path exists"
else:
   print "Minimum Cost:", cost[source]

   current = source

   while current != destination:
      print current
      current = next[current]

   print destination
```

Note: This algorithm assumes that edges connect consecutive stages and that the destination belongs to the final stage. The implementation must also validate the input graph and handle vertices that cannot reach the destination.

9. Handling Dynamic Route Changes and Batch Requests

In real-world logistics, route costs may change because of traffic congestion, adverse weather, fuel consumption, or road closures.

The system can accommodate these changes by updating the weight of an existing edge and recalculating the minimum-cost route using Dynamic Programming.

For example, if the cost of a road increases because of traffic congestion, the algorithm can identify an alternative route if one provides a lower total cost.

The system can also process multiple delivery requests in a batch. Each request specifies a source and destination, and the algorithm calculates the corresponding minimum-cost route when the request satisfies the graph's stage constraints.

Important considerations:

- A road closure can be represented by removing the corresponding edge or marking it unavailable.
- Traffic and weather conditions can be represented through updated edge weights.
- If fuel efficiency is the objective, edge weights can represent fuel consumption.
- If multiple objectives must be considered, such as time and fuel consumption, an appropriate combined cost function is required.
- For very large networks, adjacency lists and efficient input processing help reduce memory consumption.

A change in an edge weight does not automatically update previously calculated results. The program must recalculate the affected costs or rerun the algorithm before using the updated route.

10. Time and Space Complexity

Let:

- (V) = Number of vertices.
- (E) = Number of edges.

Measure	Complexity	Explanation
Time Complexity	(O(V+E))	Each vertex and edge is processed a constant number of times during the backward DP computation.
Space Complexity	(O(V)) auxiliary space	The cost[], next[], and related vertex-based arrays require linear space.
Graph Storage	(O(V+E))	An adjacency-list representation stores vertices and edges efficiently.
Total Space Complexity	(O(V+E))	Includes graph storage and the auxiliary arrays.

The (O(V+E)) time complexity applies when the graph is processed using an adjacency-list representation and its vertices are processed in reverse topological or stage order.

The algorithm is suitable for large multistage networks containing thousands of vertices and edges, provided that the graph is represented and processed efficiently.

11. Conclusion

The Multistage Graph problem demonstrates how Dynamic Programming can be used to solve route-optimization problems efficiently.

The transportation network is represented as a directed, weighted graph divided into stages. By calculating minimum costs in reverse stage order and storing the next vertex on the optimal route, the algorithm identifies a minimum-cost path without examining every possible route.

The approach supports path reconstruction, route-cost updates, and batch processing of delivery requests. With an efficient graph representation, the algorithm operates in (O(V+E)) time and requires (O(V+E)) total space, making it suitable for large logistics networks.

Thus, Dynamic Programming provides an effective method for optimizing delivery routes and improving transportation efficiency in the SwiftCargo scenario.
